SIMATSENGINEERING(SAVEETHASCHOOLOFENGINEERING)

DepartmentofArtificialIntelligenceandDataScience

ASSESSMENTTOOL1 –Experiments

CourseCode:	DSA03	CourseTitle:	NaturalLanguageProcessing
COAssessed:	CO3 – Precision	Assessment:	AssessmentTool1–Experiments
Weightage:	50% of		
CO5Statement:	ApplyN-grammodels,smoothingtechniques,part-of-speechtaggingsystemsandt oevaluate effective syntactic analysis. (BL-3)		
Student Name:	Y.Harsha Vardhan Re	Reg. No.:	192425227
Section/Batch:	B.TechAI-ML	Date:	08-08-2026

CodeofConduct

I Y.Harsha Vardhan Reddy (Name /Reg No) certify that thissubmission is myoriginalworkandthatIhaveadheredtotheguidelinesspecifiedforthisassessment.Iunderstandthatanyviolationofacademicintegrity rules will result in disciplinary action.

Signature of theStudent:

Instructions

- PerformallexperimentsusingGoogleColab.
- Createaseparatenotebook(.ipynb)fortheassessment.
- Ensureallcodeisexecutablewithouterrors.
- Includeappropriatecommentsexplainingthelogicused.
- CompleteallthreeexperimentsinasingleGoogleColabnotebook.
- Executeallcellsanddisplaytheoutputsforthegiventestcases.
- Savethenotebookwiththenamingconvention:RegNo_Name_CO3-AT1.ipynb • UploadthenotebooktoyourGitHubrepository.
- Ensuretherepositoryispublicoraccessibletotheinstructor.
- Submitthefollowing:
- GitHubRepositoryLinkinGCR
- GoogleColabShareLinkinGCR
- Screenshotsofuploadedcodingandoutputfilesasprintout.
- Duration:30minutes.

Context-Free Grammars for EnglishSyn Context-Free Rules andTrees,Sentence-LevelConstructions ng, Top-Down Parsing, EarleyParsing	Design a Python programsentences usin production andconstructparsetreerepresentatio ns for valid sentences.	Q1 to validate rules

Agreement,FeatureStructures,SubCategorization	Design a Python programsubject–verb agreement andgrammatical constraints usingstructures and subcategorization	Q2 to verify validate feature rules.
ProbabilisticContext-FreeGrammars(PCFGs)	Design a Python programsentences and compute sentenceprobabilitiesusingprobabilisticgrroduction rules.	Q3 to parse

Experiment1:CFGRuleValidationandParseTreeConstruction

ThegivenCFGisS→NPVP,NP→DetN,andVP→ VNP,withdeterminersthe/a,nouns student/teacher/book, and verbs reads/likes.

Aim

Todesign a Python program to validate whethera givenEnglish sentencefollowsthespecifiedContext-Free Grammar (CFG) and construct its parse tree.

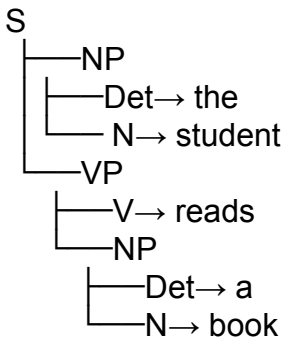
Concept

AContext-FreeGrammar(CFG)definesthesyntacticstructureofalanguageusingproductionrules. Here, a valid sentence must follow:

S→ NPVP
NP→DetN
VP→ VNP
Det→ the |a
N→student |teacher|book
V→reads|likes

Therefore,thesentencestructureis:
Determiner+Noun+Verb+Determiner+Noun

Example:
thestudentreadsabook



Algorithm

1. Acceptasentencefromtheuser.
2. Splitthesentenceinto words.
3. Checkwhetheritcontainsexactlyfivewords.
4. CheckeachwordaccordingtotheCFGrules.

5. Ifallrulesmatch,constructtheparsetree.
6. Otherwise,return InvalidSentence.

```
PythonProgram
defparse_cfg(sentenc
   e): w =
   sentence.split()
   iflen(w)!=5:
      return"InvalidSentence"
   det = ["the", "a"]
   noun=["student","teacher","book"]
   verb = ["reads", "likes"]
   if(w[0]indetandw[1]innounand w[2]
      in verb and w[3] in det and w[4]
      in noun):
      returnf"""S
 ├──NP
  │ ├──Det→ {w[0]}
   │ └── N→{w[1]}
 └──VP
     ├──V→{w[2]}
    └──NP
       ├──Det→ {w[3]}
       └──N→{w[4]}"""

   return"InvalidSentence
" tests = [
   "thestudentreadsabook",
   "a teacher likes the
   book", "student reads
   book",
   "thebooklikesateacher",
   "reads the student
   book"
]
for sentence in tests:
   print("\nSentence:",senten
   ce)
   print(parse_cfg(sentence))
```

ExpectedOutput

```
Sentence:thestudentreadsaboo
k Valid Parse Tree

Sentence:ateacherlikesthebook
Valid Parse Tree

Sentence:studentreadsbook
Invalid Sentence

Sentence:thebooklikesateacher
Valid Parse Tree

Sentence:readsthestudentbook
```

Invalid Sentence
Thesevalidity resultsmatch thetest casesspecified intheassessment.

Result

Thus,thePythonprogramsuccessfullyvalidatessentencesusingCFGproductionrulesandgenerate sa parse tree for grammatically valid sentences.

Experiment2:AgreementandFeatureStructureChecking

The experiment requires checking subject–verb agreement using Number and Personattributes.The supplied rules classify he, she, and it as singular third person, while they is plural.

Aim
ToimplementaPythonprogramthatverifiessubject–verb agreementusingfeaturestructures.
Concept
Subject–verbagreementmeansthattheverbmustagreewithitssubjectingrammaticalfeaturessuch as number.

Forexample:
he runs→
Correcthe run →
Incorrect they
run→ Correct
theyruns→Incorrec
t Feature structures:
he→singular,thirdperson
she→singular,thirdperson
it→ singular, third person
they → plural
runs,writes→singularverbs
run, write→ plural verbs

Algorithm

1. Acceptthesubjectandverb.
2. Storesubjectfeaturesin adictionary.
3. Storeverbfeaturesinanotherdictionary.
4. Obtainthenumberfeatureof thesubject.
5. Obtainthenumberfeatureof theverb.
6. Comparethetwo features.
7. ReturnTruewhentheyagree;otherwisereturn False.

PythonProgram

```
defcheck_agreement(subject,verb):

   subjects={
      "he": "singular",
      "she":"singular",
      "it":"singular",
      "they":"plural"
   }

   verbs={
      "runs": "singular",
```

```
    "writes":"singular",
    "run": "plural",
    "write":"plural"
  }

  ifsubjectnotinsubjectsorverbnotinverbs:
    return False

  returnsubjects[subject]==verbs[verb]


tests=[
  ("he","runs"),
  ("he","run"),
  ("they","run"),
  ("they","writes"),
  ("she","writes")
]

for subject, verb in tests:
  print(subject,verb,"→",
      check_agreement(subject,verb))
```

Output
he runs →
True he run →
False
theyrun→ True
theywrites→False
she writes → True
Thesearethe expected agreementresultsgivenintheassignment.

Result
Thus,theprogramsuccessfullyverifiessubject–verbagreementbycomparinggrammaticalfeature structures of subjects and verbs.

Experiment3:ProbabilisticContext-FreeGrammar(PCFG)Parsing

ThesuppliedPCFGassignsprobabilitiesJohn=0.6,Mary=0.4,and runs/walks=0.5,whileS→NPVP has probability 1.0.

Aim
ToimplementaPythonprogramtoparseasentenceandcalculateitsprobabilityusingaProbabilisti c Context-Free Grammar (PCFG).

Concept
AProbabilisticContext-FreeGrammar(PCFG)assignsaprobabilitytoeachgrammarproduction. Given grammar:
S→ NPVP [1.0]

NP → John [0.6]
NP→ Mary [0.4]

VP → runs [0.5]

VP→ walks [0.5]
Thesentenceprobabilityiscalculatedas:
P(Sentence)
=P(S→NPVP)×P(NP)×P(VP)
Forexample:
John runs

P =1.0 ×0.6 ×0.5

P =0.30
Algorithm
1. Acceptatwo-wordsentence.
2. Splititinto noun and verb.
3. Checkwhether thenounexistsintheNPgrammar.
4. Checkwhether theverbexistsin theVPgrammar.
5. Retrievetheircorresponding probabilities.
6. Multiplythegrammarprobabilities.
7. Return0.00whenthe sentencecannotbe generated.

PythonProgram

```
defsentence_probability(senten
   ce): words = sentence.split()
   iflen(words)!=2:
      return 0.0
   np_prob =
      { "John":
      0.6,
      "Mary":0.4
   }
   vp_prob =
      { "runs":0.5,
      "walks":0.5
   }
   noun,verb =words
   ifnounnotinnp_proborverbnotinvp_prob:
      return 0.0
   return1.0*np_prob[noun]*vp_prob[verb]
tests = [
   "John runs",
   "John walks",
   "Mary runs",
   "Marywalks",
   "Peter runs"
]
for sentence in tests:
   print(sentence,"→
   ",
      f"{sentence_probability(sentence):.2f}")
```

Output

```
John runs → 0.30
John walks → 0.30
Mary runs → 0.20
Marywalks→0.20
```

Peter runs → 0.00
Theseprobabilitiescorrespondexactlytothetestcasesintheprovided assessment.

SampleCalculations
1. Johnruns
P(John runs)
=P(S→NPVP)×P(NP→ John)×P(VP→ runs) =1.0 ×0.6 ×0.5
=0.30
2. Marywalks
P(Marywalks)
=1.0 ×0.4 ×0.5
=0.20
3. Peterruns
Peterisnotpresentinthe NPproductionrules.
Therefore,
P(Peterruns)=0.00

Result
Thus,thePCFGprogramsuccessfullyparsesvalidsentencesandcalculatestheirprobabilitiesusingpr obabilistic production rules. Invalid sentences are assigned a probability of 0.00.

OverallConclusion

The three experiments demonstrate important concepts of Natural Language Processingandsyntactic analysis.Experiment1usesCFGproductionrulestovalidatesentencestructureandconstructparsetr ees.
Experiment2appliesfeaturestructurestochecksubject–verbagreement.Experiment3extendsCFGwith probabilities using PCFG to calculate the likelihood of valid sentences. Theseexperimentsdemonstrate rule-based, feature-based, and probabilistic approaches to syntactic analysis.

RubricsforCalculation

Criteria	Weigh	Level4–Exemplary	
Understanding of Concepts	25	Demonstratesthorough understanding effectively int multiple concepts/technologies	ur m
Experimental Design	30	Well-planned,systematicimplementation optimal solution	i wit
UseofTools		Effective anduseoftools/technologhproperconfiguration	advar opriat of toolsc cution
Analysisof Results	15	Insightful analysis accurate interpre andvalidationofresults	Goo withr bleint t
Documentation		Clear,well-structureddocumentation 10 diagrams, code, andresults	Orga docu withr aryc

–

or

or

or

or

Q.No./ Criteria Level	Understandin gof Concepts	Experime ntalDesig n	Useof Tools	Analysi s of Results	Docume n tation	Sub.Total	Total %
1							
2							
3							

FacultyIn-charge	CourseCoordinator	ProgramDirector

Signature:	Signature:	Signature:
Date:	Date:	Date: